# 软件需求分析与设计

## 第五章框架梳理

仅供学习参考

# 目 录


**一、 高级类建模 - 1 -**

（一） 扩展机制 - 1 -

1. 构造型（Stereotype） - 1 -

2. 注释 - 1 -

3. 约束 - 1 -

4. 标签 - 1 -

（二） 可见性与封装 - 1 -

1. 可见性分类 - 1 -

（三） 导出信息（Derived Information） - 2 -

1. 定义和性质 - 2 -

（四） 限定关联（Qualified Association） - 2 -

1. 定义和性质 - 2 -

（五） 关联类和具体化类 - 2 -

1. 定义和性质 - 2 -

**二、 高级泛化与继承建模 - 3 -**

（一） 泛化和可替换性 - 3 -

1. 泛化的定义 - 3 -

2. 泛化的作用 - 3 -

（二） 继承与封装 - 3 -

1. 继承（inheritance）的定义 - 3 -

2. 封装（encapsulation）的定义 - 3 -

3. 继承与封装的关系 - 3 -

（三） 接口继承 - 3 -

1. 接口继承的定义 - 3 -

2. 接口与抽象类的区别 - 3 -

（四） 实现继承 - 3 -

1. 扩展（extension）继承——恰当使用 - 3 -

2. 限制（restriction）继承——有问题的使用 - 4 -

3. 方便（convenience）继承——不恰当的使用 - 4 -


仅供学习参考

4. 实现继承的危害 - 4 -
**三、 高级聚合和委托建模 - 5 -**
（一） 聚合的更多语义 - 5 -
1. Exclusive Owns 聚合——换了主人不能存活 - 5 -
2. Owns 聚合——换主人也能存活 - 5 -
3. Has 聚合 - 5 -
4. Member 聚合 - 5 -
（二） 委托和原型模型 - 5 -
1. 委托（delegation）和原型系统（Prototypical Systems） - 5 -
2. 委托和继承 - 5 -
3. 整体构件（Holon） - 5 -
**四、 高级交互建模 - 6 -**
（一） 生命线 - 6 -
1. 生命线的定义 - 6 -
2. 生命线的命名 - 6 -
3. 对象（生命线）的销毁 - 6 -
（二） 消息 - 6 -
1. 消息的类型 - 6 -
（三） 片段 - 7 -
1. 片段的定义 - 7 -

仅供学习参考

# 一、高级类建模

## （一）扩展机制

### 1. 构造型（Stereotype）

(1) 表示方法：<<global>>，<<include>>

(2) 构造型对现有的 UML 建模元素进行扩展，使现有元素的语义多样化

(3) 构造型扩展的是语义而不是 UML 结构

### 2. 注释

(1) 表示方法：右上角卷起的矩形 

(2) 在 UML 的任何位置都能插入注释

(3) 是对建模决策的附加说明，对模型语义没有任何影响

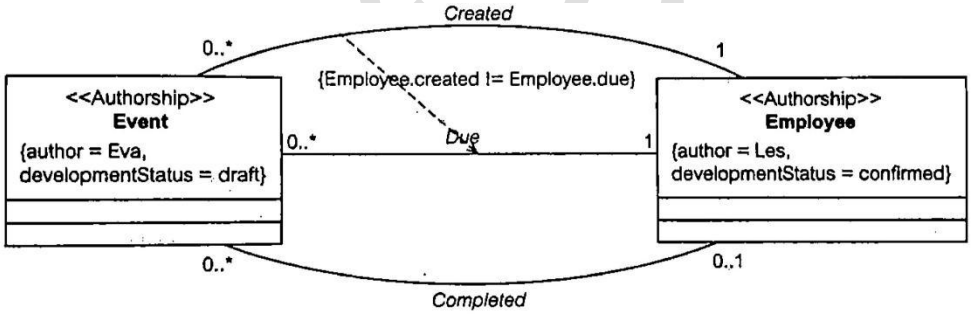
### 3. 约束

(1) 表示方法：在对象旁边添加大括号 {}

(2) 指条件或限制，是对一个元素某些语义的声明

(3) 对模型具有语义含意，并且用形式化的约束语言来描述

(4) 关联约束：在关联线（虚箭头）上增加一个约束，约束被绑定在该箭头上




关系管理系统中关联的约束

### 4. 标签

(1) 标签定义：构造型的一个特性，显示为含有构造型声明的类矩形中的一个属性

(2) 标签值：是一个“名—值”对，附属于一个使用了包含标签定义的构造型的模型元素

(3) 类似于 C 语言中的结构体类型，标签必须先定义后使用

## （二）可见性与封装

### 1. 可见性分类

(1) 四种可见性标志（相关继承和可见性特性均与 Java 相同）

① 公共可见性（+）——public

② 私有可见性（-）——private

③ 保护可见性（#）——protected

④ 包可见性（~）——package

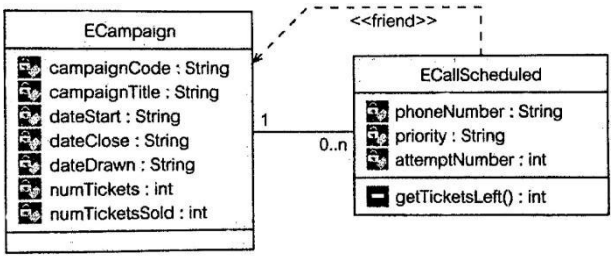
(2) 友元可见性

① 友元关系可以是另一个类或另一个类的操作

② 友元关系是**单向**的（A 让 B 成为友元，但 B 不一定是 A 的友元）

③ 友元关系要在**授予**友元关系的类中声明

④ 表示方法：用虚线箭头从一个友元类指向授予友元关系的类，并标注**<<friend>>**




电话销售系统中的友元

（三）导出信息（Derived Information）

1. 定义和性质

(1) 是一种经常应用于属性或关联的约束

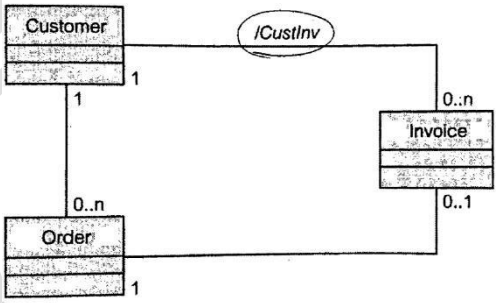
(2) 导出信息是**冗余**的——它可以在需要时计算出来，使模型更容易可读

(3) 在设计模型中，需要考虑信息存取的最优化问题，所以了解哪些信息使导出的很重要

(4) 表示方法：在导出属性名或关联名前面加一条**斜线“/”**

(5) **导出属性**：不写也可以算出的属性，写出来更清晰明确

(6) **导出关联**：并不推荐，因为导出后形成了关联的回路，需要去除一个不重要的原生关联




导出关联

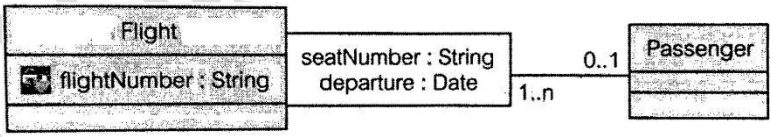
（四）限定关联（Qualified Association）

1. 定义和性质

(1) 在二元关联的“一对多”的“多”的一端增加**关联的属性**

(2) 将多对多的关联细化成一对多或一对一的关联，降低关联**多重度**

(3) 关联本身具有**限定符**（有属性），一般**不采用双边**限定



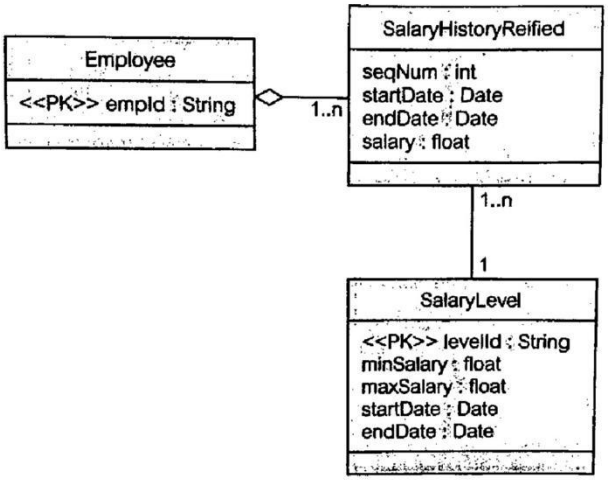

限定关联

（五）关联类和具体化类

1. 定义和性质

(1) 带关联类的模型更像是 E-R 图的多对多联系

(2) 带有具体化类的模型能够**无语义损失**地代替关联类




使用具体化类的更好的解决方案

# 二、高级泛化与继承建模

## （一）泛化和可替换性

### 1. 泛化的定义

(1) 泛化是一种静态状态，是类之间的语义关系

(2) 它说明子类的接口必须包含超类的所有（public，package，protected）特性

### 2. 泛化的作用

(1) 利用**可替换性原则（Substitutability principle）**减少模型中关联关系和聚合关系的总数

① 可替换性原则：在代码中任何访问超类对象的地方，都可以用子类对象来替换超类对象

(2) 提高系统模型的表现力、可理解性和抽象程度

## （二）继承与封装

### 1. 继承（inheritance）的定义

(1) 子类可以去访问父类的属性和方法，允许子类直接访问 protected 属性

(2) 继承是一种机制，通过继承将较特殊的元素合并较一般元素中定义的结构和行为

### 2. 封装（encapsulation）的定义

(1) 基于类的封装，只有通过对象接口中的操作才能访问对象的状态（属性值）

### 3. 继承与封装的关系

(1) 继承与封装的查询能力是正交的、矛盾的，它不得不与这两个特性一起折中考虑

## （三）接口继承

### 1. 接口继承的定义

(1) 以可替换性为目标来使用泛化

(2) 同义词：子类型、类型继承

### 2. 接口与抽象类的区别

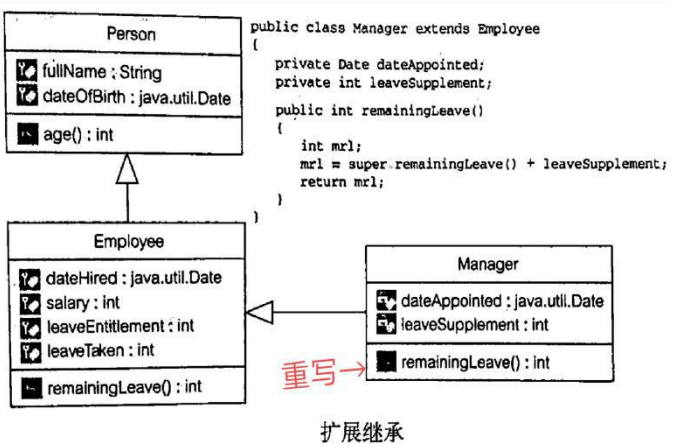
(1) 抽象类：类中存在至少一个抽象方法，可以存在已经被实现的方法

(2) 接口：类中的所有方法都是抽象方法，接口中只能有常量

## （四）实现继承

### 1. 扩展（extension）继承——恰当使用

(1) 将继承作为类的增量式定义，子类具有比超类更多的特性（属性、方法）

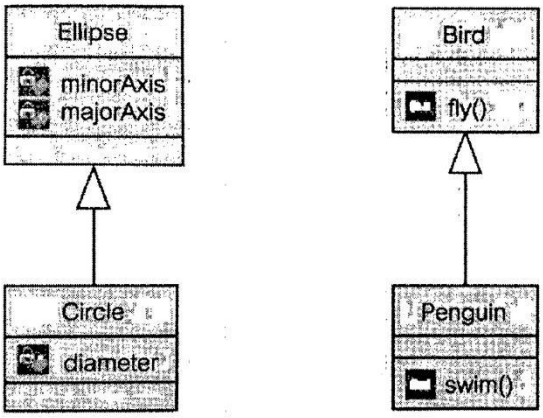



扩展继承

仅供学习参考

2. 限制（restriction）继承——有问题的使用

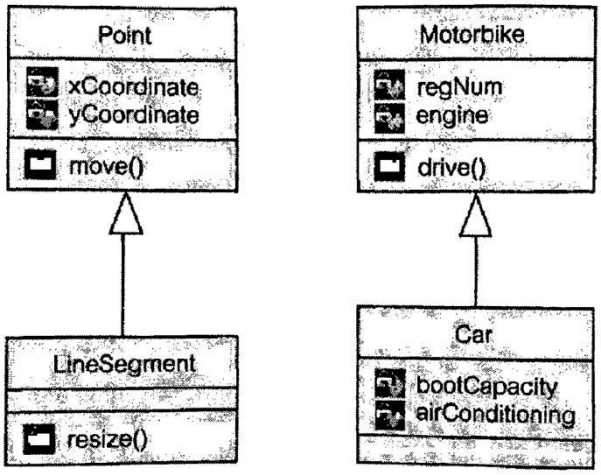
(1) 继承来的特性在子类中被禁止，甚至被覆盖




限制继承

3. 方便（convenience）继承——不恰当的使用

(1) 任选一个类作为其他类的父类（随便继承）




方便继承

4. 实现继承的危害

(1) 脆弱的基类（Fragile base class）

① 问题：允许对（一个或多个）超类的实现进行演化的同时，使其子类仍然有效并可用

② 后果：父类属性方法的改变会影响到子类

③ 解决：将公共接口声明为不可变的，或避免对控制范围之外的超类进行实现继承

(2) 重载、向下调用和向上调用

① 重载：子类重写父类中的某个方法，但是参数列表不相同

② Java 中使用 super 向上调用

③ Java 中若子类有重载方法，那么当构造父类对象并调用该方法时，默认使用子类中方法

(3) 多重实现继承

① 考虑了接口协议的合并，允许实现片段的合并

② 会引起复杂性的内在增长

# 三、高级聚合和委托建模

## （一）聚合的更多语义

1. Exclusive Owns 聚合——换了主人不能存活

   (1) {frozen} 约束：一个构件类对象不能再连接到另一个复合对象上

   (2) 【举例】书◆—<<Exclusive Owns>>—书中的章节、扑克牌◇—<<Exclusive Owns>>— 54 张牌

2. Owns 聚合——换主人也能存活

   (1) 一个构建对象最多属于一个复合对象，但它能被再连接到一个复合对象上

   (2) 【举例】水壶◆—<<Owns>>—水、自行车◇—<<Owns>>—轮胎

3. Has 聚合

   (1) 不具有存在依赖性，复合对象的删除不会自动向下扩散到构件对象上

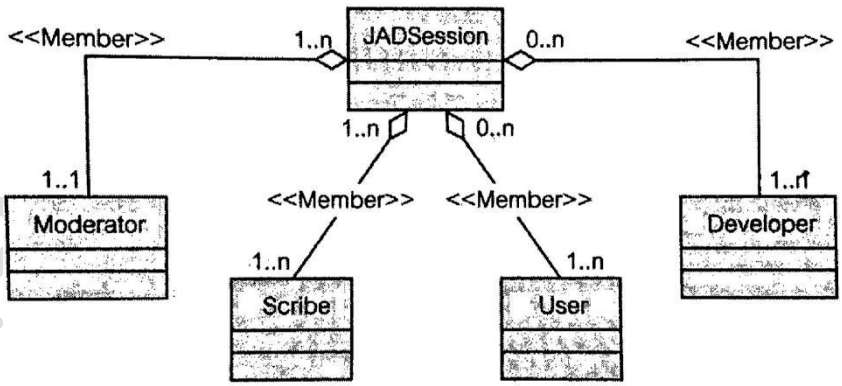
   (2) 不能使用实心菱形，因为不具有复合关系

   (3) 【举例】购物车◇—<<Has>>—啤酒箱◇—<<Has>>—啤酒瓶（传递性、非对称性）

4. Member 聚合

   (1) 具有多对多的多重性

   (2) 【举例】：




Member聚合

## （二）委托和原型模型

1. 委托（delegation）和原型系统（Prototypical Systems）

   (1) 复合对象（外部对象）通过访问它的构建对象（内部对象）中的方法，来完成自身不能完成的任务

   (2) 一个对象和与系统中任何其他可标识的及课件的对象之间都可以有委托关系

   (3) 从已有的原型对象中构造出来的系统称为原型系统

2. 委托和继承

   (1) 委托可以对继承建模，继承也可以对委托建模，继承或委托可以用来表达相同的系统功能

   (2) 在继承中，服务完成后总是将控制返回给接收原始消息的对象，而在委托中必须明确声明

      ① i.e. 在委托中，任何自递归都必须明确地计划并设计成委托；在实现继承中，自递归总会出现

3. 整体构件（Holon）

   (1) 既表现出部件地相互依赖性，又表现出整体的独立特性（既是部分对象又是整体对象）

仅供参考

# 四、高级交互建模

## （一）生命线

1. 生命线的定义

   (1) 代表一个交互参与者，标识在交互过程中一个特定时间段内，对象是存在的

2. 生命线的命名

   (1) 一个类的未命名实例——:Class1

   (2) 一个类的命名实例——c2:Class2

   (3) 一个类，即元类（meta-class）的一个实例——:Class 3——显示对类自身静态方法的调用

   (4) 一个接口——:Interface1

3. 对象（生命线）的销毁

   (1) 在图上用“X”来表示，标记在生命线的末端

   (2) 执行中创建的对象要在程序结束前销毁（malloc 和 free）

## （二）消息

1. 消息的类型

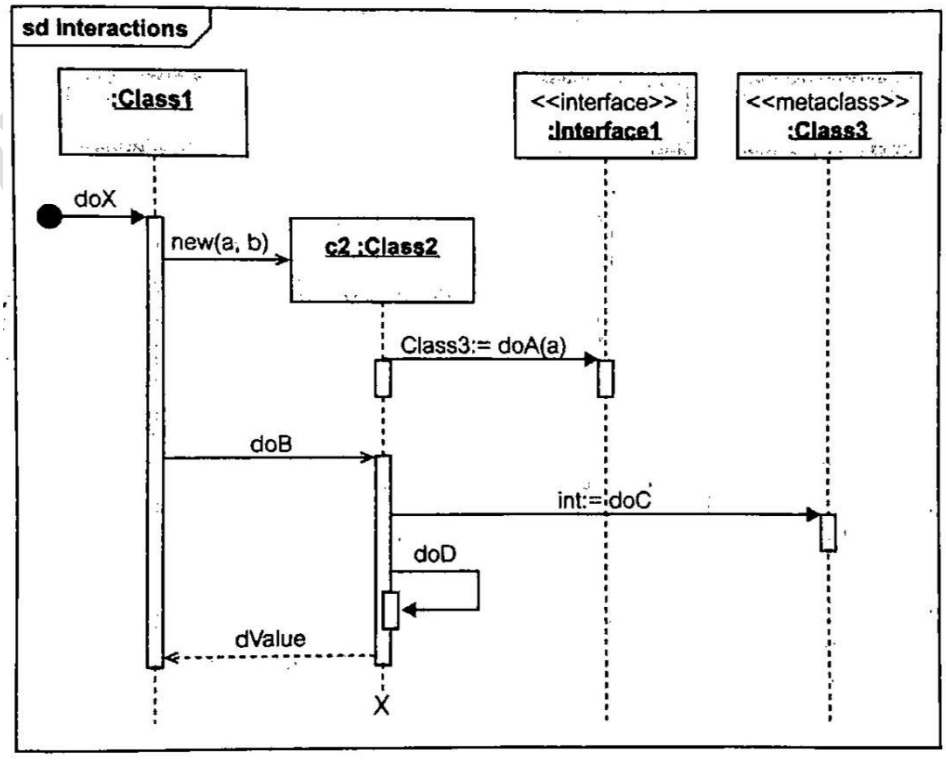
   (1) 同步（synchronous）消息 →：调用者阻塞，等待响应

   (2) 异步（asynchronous）消息 →：调用者不阻塞（多线程执行）

   (3) 对象创建消息 —new(a, b)→：创建一个对象，是特殊的异步消息

   (4) 回复消息 - - -→：将交互的输出值传递给发起动作的调用者，并标有返回值的描述

   (5) 发现（found）消息 ●→：表示没有指定发送者的消息，发送消息的来源在模型表示的范围之外




生命线和消息

（三）片段

1. 片段的定义

(1) 一段交互称为**交互片段**

(2) 交互可以包含更小的交互片段，称为**组合片段**

(3) 组合片段的语义由**交互操作符**（interaction operator）确定

(4) 片段的分类

① 可选片段（alt）：在守卫条件中表达 if-then-else 条件逻辑

② 选择片段（opt）：如果守卫条件为真，执行该片段

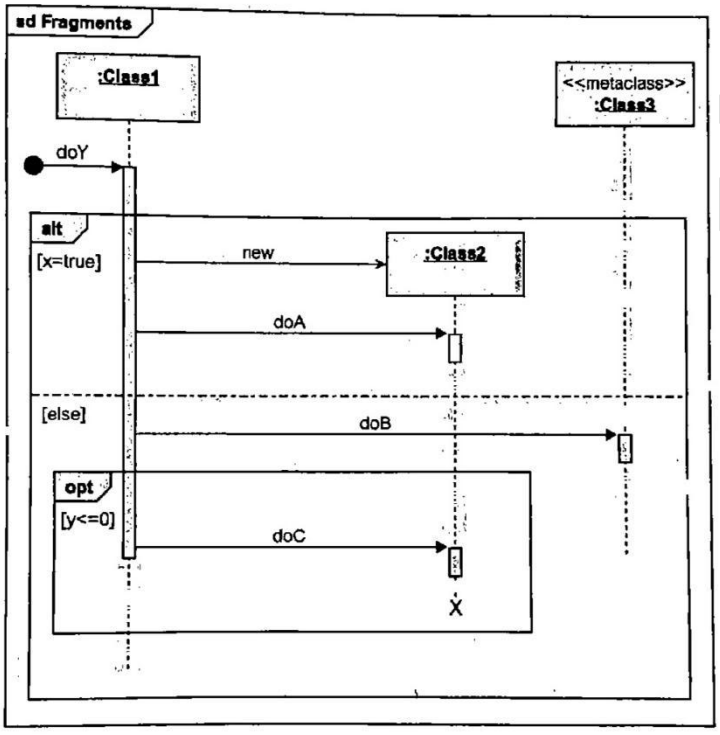
③ 循环片段（loop）：服从循环条件而重复多次的片段

④ 中断片段（break）：如果中断条件为真，就执行中断片段，而不执行外围片段剩余的部分

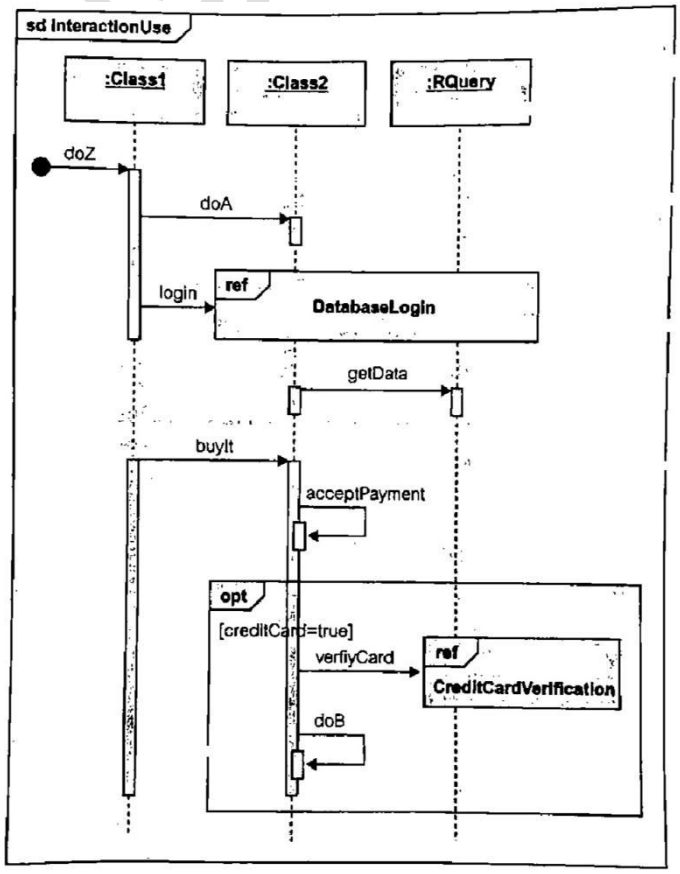
⑤ 并行片段（parallel）：允许所包含行为交替进行

(5) 交互使用

① 一个外围交互（sd）对另一个交互（ref）的引用称为交互使用




片段




交互使用